□ PayFlow API

Complete Developer Guide & Architecture Report

A Production-Grade FinTech REST API

Node.js • Express • PostgreSQL • Redis • BullMQ • Prisma • JWT • Zod

1. What Is This Project?

PayFlow API is the backend engine of a FinTech payment application — similar to how Revolut, PayPal, or CashApp works under the hood. There is no visual frontend. Instead, this is a server that other apps (mobile apps, websites) communicate with through HTTP requests to move money, manage wallets, and track transactions.

Think of it this way:

Real-World Analogy
Your phone app (Revolut) = the frontend (not built here)
PayFlow API = the backend server we built
PostgreSQL database = the vault where all data lives
Redis = a super-fast memory layer for queues
BullMQ = the background worker that processes transfers

When you tap 'Send £50 to Bob' on a payment app, behind the scenes the app sends an HTTP request to an API exactly like this one. The API validates your identity, checks your balance, queues the transfer, debits your wallet, credits Bob's wallet, and records the transaction — all within milliseconds.

1.1 What Does PayFlow API Do Specifically?

Feature	Description
User Registration	Create an account with email, password, name. A default USD wallet is auto-created.
JWT Authentication	Login returns an access token (15 min) and refresh token (7 days). Secure, stateless auth.
Token Rotation	Every refresh issues a brand new token pair and revokes the old one. Prevents token theft.
Multi-Currency Wallets	Each user can have USD, EUR, GBP, or NGN wallets. One wallet per currency per user.
Wallet Funding	Simulate depositing money into a wallet. Atomically updates balance + creates transaction record.
Wallet-to-Wallet Transfer	Send money from your wallet to another user's wallet. Idempotent and async.
Idempotency	Sending the same transfer request twice (with the same key) won't charge twice. Like Stripe.

Async Queue Processing	Transfers are queued in Redis/BullMQ. A background worker processes them with auto-retry.
Transaction History	Full paginated, filterable transaction history. Filter by status, type, currency, wallet.
Rate Limiting	Prevents brute-force attacks. Auth endpoints limited to 10 requests/15 min.
Swagger UI Docs	Auto-generated interactive API docs at /api-docs. Click to test endpoints in browser.
Health & Metrics	GET /health and GET /metrics endpoints for monitoring uptime and memory usage.

2. Tools Used & Why

Every tool in this project was chosen for a specific reason. Here is a plain-English explanation of each one.

2.1 Node.js — The Runtime

Node.js lets you run JavaScript on a server (not just in a browser). It's extremely fast for handling many simultaneous requests because it uses an event-driven, non-blocking model — meaning it doesn't wait for one task to finish before starting another. This is perfect for a payment API that handles thousands of concurrent users.

2.2 Express.js — The Web Framework

Express is the layer that receives HTTP requests (GET, POST, etc.) and routes them to the right code. Without Express, you'd have to manually parse every URL and request body. Express gives you a clean, organised way to define routes and middleware. Think of it as the traffic controller of the API.

2.3 PostgreSQL — The Database

PostgreSQL is a powerful, battle-tested relational database. We chose it over simpler options like MongoDB because financial data is relational by nature — users have wallets, wallets have transactions. In a relational database, these relationships are enforced by the database itself, preventing orphaned or corrupted data. It also supports ACID transactions — meaning either ALL of a transfer happens, or NONE of it does. You'll never end up with money debited but not credited.

2.4 Prisma — The ORM (Object Relational Mapper)

Prisma lets you interact with PostgreSQL using JavaScript objects instead of writing raw SQL. You define your database structure in a schema file (prisma/schema.prisma), and Prisma generates type-safe code to query it. It also handles database migrations — tracking changes to your database structure over time, like version control for your database.

Without Prisma vs With Prisma
Without Prisma (raw SQL): db.query('SELECT * FROM users WHERE id = $1', [userId]) With Prisma: prisma.user.findUnique({ where: { id: userId } })
Prisma is safer, more readable, and catches errors at code-write time, not runtime.

2.5 Redis — The In-Memory Store

Redis stores data in RAM (memory) instead of on disk. This makes it 10-100x faster than PostgreSQL for certain operations. In PayFlow, Redis serves two purposes:

- Queue storage — BullMQ stores job data (pending transfers) in Redis
- Token blacklist — Revoked refresh tokens are tracked in Redis for instant lookup

Redis is why you can check 'is this token revoked?' in under 1 millisecond instead of hitting the database every time.

2.6 BullMQ — The Job Queue

BullMQ is a production-grade queue library built on top of Redis. When a user initiates a transfer, we don't process it immediately in the HTTP request. Instead, we create a 'job' and add it to the BullMQ queue. A separate background worker picks it up and processes it. This is exactly how real payment systems work.

Why Use a Queue for Transfers?
1. Resilience — if the server crashes mid-transfer, the job stays in the queue and retries
2. Speed — the API responds immediately (202 Accepted) without waiting for DB updates
3. Retry logic — failed jobs automatically retry 3 times with exponential backoff
4. Scalability — you can add more workers to process more transfers in parallel
5. Decoupling — the API and the processor are independent; one can scale without the other

2.7 JWT — JSON Web Tokens (Authentication)

JWT is the standard way to handle authentication in REST APIs. When you login, the server creates two tokens:

- Access Token (15 min) — sent with every API request in the Authorization header to prove identity
- Refresh Token (7 days) — used only to get a new access token when the access token expires

PayFlow implements refresh token rotation — every time you refresh, the old token is revoked and a new pair is issued. This limits the damage if a token is ever stolen.

2.8 Zod — Input Validation

Zod validates every piece of data that comes into the API before it touches the database. If someone sends a negative transfer amount, an invalid email, or a missing field, Zod catches it and returns a clear error message before any damage can be done. It's the API's bouncer.

2.9 Bcrypt — Password Hashing

Passwords are never stored as plain text. Bcrypt converts passwords into a scrambled, irreversible hash. Even if the database were stolen, attackers couldn't recover the original passwords. The 'rounds' setting (12 in our config) controls how computationally expensive the hashing is — making brute-force attacks impractically slow.

2.10 Helmet — HTTP Security Headers

Helmet automatically sets secure HTTP response headers that protect against common web vulnerabilities like XSS (Cross-Site Scripting), clickjacking, and MIME sniffing. One line of code adds a dozen security protections.

2.11 Morgan — HTTP Request Logging

Morgan logs every HTTP request that comes into the server — the method, URL, status code, and response time. This is essential for debugging and monitoring in production. Logs are fed into Winston for structured output.

2.12 Winston — Structured Application Logging

Winston is the main logger for application events — database connections, errors, business events like 'user registered' or 'transfer completed'. It writes structured JSON logs in production (machine-readable) and colourful human-readable logs in development.

2.13 Swagger / OpenAPI — API Documentation

Swagger automatically generates a beautiful interactive documentation page from comments in the route files. At /api-docs, developers can read about every endpoint, see example requests, and even test them live in the browser — which is exactly what you're doing at localhost:3000/api-docs right now.

3. Database Design

The database has 4 tables. Each table maps directly to a real-world concept in the payment system.

3.1 The Tables and Their Purpose

Table	Purpose
users	Stores account information — email, hashed password, name, KYC status
wallets	Each row is one currency wallet belonging to one user
transactions	Every financial movement — credits, debits, transfers — is recorded here forever
refresh_tokens	Tracks issued refresh tokens so they can be revoked on logout or rotation

3.2 Table: users

```
Column        Type        Description
──────────────────────────────────────────────────
id            UUID        Unique identifier (auto-generated)
email         String      Unique. Used for login.
password_hash String      Bcrypt hash. NEVER the plain password.
first_name    String      User's first name
last_name     String      User's last name
phone         String?     Optional. Unique if provided.
status        Enum        ACTIVE | SUSPENDED | PENDING_KYC
kyc_status    Enum        UNVERIFIED | PENDING | VERIFIED | REJECTED
created_at    DateTime    Auto-set on creation
updated_at    DateTime    Auto-updated on every change
```

Why UUID instead of auto-increment numbers? UUIDs are unpredictable — an attacker cannot guess other user IDs by incrementing. With auto-increment, if your ID is 42, you know user 41 and 43 exist.

Why store kyc_status? KYC = Know Your Customer. Real financial apps legally require verifying user identity before allowing large transfers. Even though we don't implement the full KYC verification flow here, having the field shows architectural maturity.

3.3 Table: wallets

```
Column      Type      Description
──────────────────────────────────────────────────
id          UUID      Unique identifier
user_id     UUID      Foreign key → users.id
currency    Enum      USD | EUR | GBP | NGN
balance     Decimal   18 digits, 2 decimal places (e.g. 99999999999999.99)
```

```
status      Enum      ACTIVE | FROZEN | CLOSED
created_at  DateTime  Auto-set
updated_at  DateTime  Auto-updated

UNIQUE constraint: (user_id, currency)
  → One user cannot have two USD wallets
```

Why Decimal and not Float? Floating-point numbers have rounding errors. 0.1 + 0.2 = 0.30000000000000004 in most languages. For money, you NEVER use float. Decimal stores exact values. This is a critical financial systems principle.

Why 18,2 precision? 18 digits total, 2 after the decimal point. This supports values up to 9,999,999,999,999,999.99 — more than enough for any currency balance.

3.4 Table: transactions

```
Column             Type       Description
──────────────────────────────────────────────────────────────
id                 UUID       Unique transaction ID
idempotency_key    String     UNIQUE. Prevents duplicate processing.
sender_wallet_id   UUID?      Nullable → CREDIT transactions have no sender
receiver_wallet_id UUID?      Nullable → DEBIT transactions have no receiver
amount             Decimal    Always positive. Direction = type field.
currency           Enum       USD | EUR | GBP | NGN
type               Enum       CREDIT | DEBIT | TRANSFER
status             Enum       PENDING → PROCESSING → COMPLETED | FAILED
description        String?    Human-readable note (e.g. 'Paying Bob back')
metadata           JSON?      Flexible field for future extra data
failure_reason     String?    Set if status = FAILED
processed_at       DateTime?  When the transfer actually completed
created_at         DateTime   When the request was received
```

Why is idempotency_key unique? This is the key to preventing double-charges. When you initiate a transfer, you include a unique Idempotency-Key header. If the same key comes in again (because your network retried), the API returns the original transaction instead of creating a new one. Stripe invented this pattern and it's now industry standard.

Why track status with multiple states? A transfer goes through a lifecycle: PENDING (queued), PROCESSING (worker picked it up), then either COMPLETED (success) or FAILED (error). This gives you full visibility and lets you implement reversal/refund logic later.

3.5 Table: refresh_tokens

```
Column      Type      Description
──────────────────────────────────────────────
id          UUID      Unique identifier
user_id     UUID      Foreign key → users.id
token       String    UNIQUE. The actual JWT string.
expires_at  DateTime  When this token expires
revoked     Boolean   True if logged out or rotated
```

```
created_at   DateTime   When issued
```

Why store refresh tokens in the database? JWTs are self-contained and can't be 'cancelled' without a server-side blacklist. By storing them in the database, we can mark them as revoked on logout. This solves the classic JWT security problem.

3.6 Table Relationships

```
users ──────< wallets          (one user has many wallets)
users ──────< refresh_tokens   (one user has many sessions)
wallets ─────< transactions     (as sender OR receiver)

Example query — get all transactions for a user:
  Find all wallets WHERE wallet.user_id = user.id
  Find all transactions WHERE
    transaction.sender_wallet_id IN (user's wallet IDs)
    OR
    transaction.receiver_wallet_id IN (user's wallet IDs)
```

4. Project File Structure — File by File

Here is every file in the project, what it does, and why it exists. This is the mental map you need before touching any code.

4.1 Root Level Files

File	Purpose
package.json	Defines the project — name, scripts (npm run dev), and all dependencies
.env	Your secret config — database URL, JWT secrets, Redis host. NEVER commit this to GitHub
.env.example	A safe template showing what variables are needed, without real values. Commit this.
.gitignore	Tells Git what to ignore — node_modules, .env, log files
docker-compose.yml	Defines Postgres + Redis containers for easy local setup (optional with Docker)
Dockerfile	Instructions for packaging the app as a container for deployment
README.md	Your public documentation — what the project is, how to set it up, API reference

4.2 prisma/ Folder

File	Purpose
schema.prisma	THE most important file. Defines all 4 database tables, their columns, types, and relationships. Prisma reads this to generate migrations and client code.
migrations/	Auto-generated SQL files. Each migration = one version of your database schema. Never edit manually.
seed.js	Script that populates the database with test data (Alice, Bob with wallets and balances). Run once after migration.

4.3 src/config/ Folder

Config files are loaded once at startup and shared across the whole application.

File	Purpose

index.js	Reads all environment variables from .env and exports them as a typed config object. Central source of truth for all settings.
database.js	Creates and exports the Prisma client instance. Also sets up slow query logging in development.
redis.js	Creates and exports the Redis connection. Handles reconnection logic and logs connection events.
logger.js	Creates the Winston logger with two modes — colourful console in dev, structured JSON in production. Also writes to log files.
swagger.js	Defines the OpenAPI specification metadata (title, version, security scheme, reusable schemas). Swagger UI reads this.

4.4 src/middlewares/ Folder

Middleware are functions that run on every request before reaching your route handler. They form a chain — each one can either pass control to the next or reject the request.

File	Purpose
authenticate.js	Reads the Authorization: Bearer <token> header, verifies the JWT, looks up the user in the database, and attaches the user object to req.user. Any route using this middleware is protected.
validate.js	Takes a Zod schema and validates req.body (or req.query/params) against it. If validation fails, returns a 422 error with exactly which field failed and why. Never lets bad data reach the service layer.
errorHandler.js	The global error handler. Catches every error thrown anywhere in the app. Converts Prisma errors, JWT errors, and custom AppErrors into clean JSON responses with appropriate status codes.
rateLimiter.js	Two limiters — a general one (100 req/15 min per IP) and a stricter auth one (10 req/15 min). Prevents brute-force login attacks and API abuse.

4.5 src/utils/ Folder

Small, reusable helper functions used throughout the codebase.

File	Purpose
AppError.js	A custom Error class that includes statusCode, HTTP status string, and an error code. Lets you throw structured errors from anywhere: throw AppError.notFound('Wallet')
asyncWrapper.js	A function that wraps async route handlers so you don't need try/catch in every controller. Any thrown error is automatically forwarded to the global error handler.

pagination.js	Two functions — parsePagination() extracts page/limit from query params, paginatedResponse() wraps data with pagination metadata (total, totalPages, hasNextPage etc).
response.js	Helper functions for consistent API responses — sendSuccess(), sendCreated(), sendNoContent(). Ensures every success response has the same shape.

4.6 src/queues/ Folder

File	Purpose
transaction.queue.js	Defines the BullMQ Queue (where jobs are added) and the Worker (which processes them). The processTransfer function is where the actual atomic wallet debit/credit happens. Also exports startWorker() called on server startup.

4.7 src/modules/ Folder — The Core

Each module is a self-contained feature. Every module follows the same pattern: schema → service → controller → routes.

The Module Pattern (Layered Architecture)
schema.js — Zod validation rules for input data service.js — Business logic. Talks to database. No HTTP concepts here.
controller.js — Handles HTTP req/res. Calls service. Sends response.
routes.js — Wires URLs to controllers. Applies middleware.
Why this separation? So you can test the service layer without HTTP.
So business logic never leaks into route handlers.
So changing the database doesn't break the API contract.

5. How to Build This From Scratch — File Order

If you were starting this project from zero, here is the exact order to create files and why.

Phase 1 — Project Foundation

1. npm init -y — creates package.json
2. Install all dependencies — express, prisma, bcryptjs, jsonwebtoken, zod, bullmq, ioredis, winston, morgan, helmet, cors, swagger-jsdoc, swagger-ui-express, uuid, compression, express-rate-limit
3. Create .env and .env.example — define all config variables before writing any code
4. Create src/config/index.js — reads .env into a typed config object
5. Create src/config/logger.js — you need logging before anything else

Reason: You need the foundation (config + logging) before any other layer can work.

Phase 2 — Database

6. Create prisma/schema.prisma — define ALL 4 tables upfront (User, Wallet, Transaction, RefreshToken)
7. Run npx prisma migrate dev --name init — creates tables in PostgreSQL
8. Run npx prisma generate — generates the Prisma client
9. Create src/config/database.js — exports the Prisma client instance
10. Create prisma/seed.js — add test users so you can test immediately

Reason: Design your data model before writing any business logic. Every feature depends on the database shape.

Phase 3 — App Skeleton

11. Create src/utils/AppError.js — custom error class needed everywhere
12. Create src/utils/asyncWrapper.js — needed in every controller
13. Create src/utils/response.js — consistent response format
14. Create src/utils/pagination.js — needed by transaction history
15. Create src/middlewares/errorHandler.js — global error handling
16. Create src/middlewares/validate.js — input validation
17. Create src/middlewares/rateLimiter.js — rate limiting
18. Create src/app.js — wire everything together (Express setup, middleware, routes)
19. Create src/server.js — start the HTTP server, connect DB, graceful shutdown

Reason: Get a working empty server before building features. Confirm you can start the server and get a 404 response.

Phase 4 — Auth Module (Always First)

20. Create src/modules/auth/auth.schema.js — Zod schemas for register and login
21. Create src/modules/auth/auth.service.js — register, login, refresh, logout logic
22. Create src/modules/auth/auth.controller.js — HTTP handlers
23. Create src/modules/auth/auth.routes.js — wire routes
24. Register auth routes in app.js
25. Test: POST /api/v1/auth/register and POST /api/v1/auth/login

Reason: Auth is the gate to everything else. Build and test it completely before adding protected routes.

Phase 5 — JWT Middleware

26. Create src/middlewares/authenticate.js — now that auth works, build the JWT guard
27. Test: hit a protected route without a token → expect 401
28. Test: hit a protected route with a valid token → expect it to work

Phase 6 — Users Module

29. Create src/modules/users/users.service.js
30. Create src/modules/users/users.controller.js
31. Create src/modules/users/users.routes.js
32. Test: GET /api/v1/users/me with a valid token

Phase 7 — Wallets Module

33. Create src/modules/wallets/wallets.schema.js
34. Create src/modules/wallets/wallets.service.js — createWallet, getWallet, getUserWallets, fundWallet
35. Create src/modules/wallets/wallets.controller.js
36. Create src/modules/wallets/wallets.routes.js
37. Test: POST /api/v1/wallets (create EUR wallet), POST /:id/fund

Phase 8 — Queue

38. Create src/config/redis.js — Redis connection
39. Create src/queues/transaction.queue.js — BullMQ queue + worker + processTransfer function

40. Call startWorker() in server.js

Phase 9 — Transactions Module

41. Create src/modules/transactions/transactions.schema.js
42. Create src/modules/transactions/transactions.service.js — transfer (with idempotency), getTransaction, getHistory
43. Create src/modules/transactions/transactions.controller.js
44. Create src/modules/transactions/transactions.routes.js
45. Test: Full transfer flow — fund wallet, transfer, check history, test duplicate idempotency key

Phase 10 — Docs & Polish

46. Add Swagger JSDoc comments to all controller files
47. Create src/config/swagger.js
48. Wire Swagger UI into app.js
49. Add /health and /metrics endpoints
50. Write tests — unit tests for services, integration tests for routes
51. Write README.md

6. How Each API Flow Works

Let's trace exactly what happens in the code when you call each major endpoint.

6.1 POST /api/v1/auth/register

```
1. Request hits Express router → auth.routes.js
2. authLimiter middleware — checks rate limit (10 req/15 min)
3. validate(registerSchema) — Zod checks email format, password strength, name length
4. auth.controller.js register() called
5. auth.service.js register() called:
   a. prisma.user.findUnique({ where: { email } }) — check if email taken
   b. If taken → throw AppError.conflict('EMAIL_TAKEN') → 409 response
   c. bcrypt.hash(password, 12) — hash the password
   d. prisma.user.create({ data: { ...user, wallets: { create: [{ currency: 'USD' }] }
} })
      ↑ Creates user AND their default USD wallet in ONE database transaction
6. controller sends 201 Created with user data (no passwordHash exposed)
```

6.2 POST /api/v1/auth/login

```
1. validate(loginSchema) — checks email and password present
2. auth.service.js login():
   a. prisma.user.findUnique({ where: { email } })
   b. If not found → throw AppError.unauthorized('Invalid email or password')
      ↑ IMPORTANT: same message whether email wrong or password wrong (security)
   c. bcrypt.compare(password, user.passwordHash) — verify password
   d. If invalid → throw 401
   e. If user.status === 'SUSPENDED' → throw 403
   f. jwt.sign({ userId }, accessSecret, { expiresIn: '15m' }) → accessToken
   g. jwt.sign({ userId, jti: uuid() }, refreshSecret, { expiresIn: '7d' }) →
refreshToken
   h. prisma.refreshToken.create({ data: { userId, token, expiresAt } })
3. Return: { accessToken, refreshToken, user }
```

6.3 POST /api/v1/transactions/transfer (The Star Endpoint)

```
1. authenticate middleware:
   a. Extract token from Authorization: Bearer <token> header
   b. jwt.verify(token, accessSecret) — verify signature and expiry
   c. prisma.user.findUnique({ where: { id: decoded.userId } }) — confirm user exists
   d. Check user.status !== 'SUSPENDED'
   e. Attach user to req.user
2. validate(transferSchema) — Zod checks amount > 0, valid UUIDs, amount <= 1,000,000
3. Controller checks Idempotency-Key header present (required field)
4. transactions.service.js transfer():
   a. prisma.transaction.findUnique({ where: { idempotencyKey } })
      → If exists, return it immediately (DUPLICATE) — no processing
   b. prisma.wallet.findFirst({ where: { id: senderWalletId, userId } })
      → Must belong to THIS user (security check)
   c. Check senderWallet.status === 'ACTIVE'
   d. Check senderWallet.balance >= amount (sufficient funds)
   e. prisma.wallet.findUnique({ where: { id: receiverWalletId } })
   f. Check receiverWallet.status === 'ACTIVE'
   g. Check senderWallet.currency === receiverWallet.currency (no cross-currency yet)
```

```
   h. Check senderWalletId !== receiverWalletId (no self-transfer)
   i. prisma.transaction.create({ data: { ...transfer, status: 'PENDING' } })
   j. transactionQueue.add('process-transfer', { transactionId, senderWalletId, ... })
5. Return 201 with PENDING transaction — API responds IMMEDIATELY

Meanwhile, in the background:
6. BullMQ worker picks up the job from Redis queue
7. processTransfer() runs:
   a. Update transaction status → PROCESSING
   b. Re-validate wallet states and balance (could have changed)
   c. prisma.$transaction([   ← ATOMIC — all or nothing
        wallet.update({ balance: { decrement: amount } }),  ← debit sender
        wallet.update({ balance: { increment: amount } }),  ← credit receiver
        transaction.update({ status: 'COMPLETED' })
      ])
8. If any step fails → transaction status → FAILED, job retries up to 3 times
```

6.4 Idempotency in Action

Scenario: Network Timeout
User sends transfer request with Idempotency-Key: pay-bob-001 Server receives it, creates PENDING transaction, queues the job
Server sends 201 response — but the network drops!
User's app doesn't know if it worked. It retries the same request.
Server receives it again with same Idempotency-Key: pay-bob-001
Server finds existing transaction in DB → returns it immediately (200) No second transfer. No double charge. Bob gets paid exactly once.
This is why Idempotency-Key is REQUIRED on the transfer endpoint.

7. Key Database Queries Explained

Here are the most important Prisma queries in the project and what SQL they generate.

7.1 Atomic Wallet Transfer

```
// Prisma code:
await prisma.$transaction([
  prisma.wallet.update({
    where: { id: senderWalletId },
    data: { balance: { decrement: amount } }
  }),
  prisma.wallet.update({
    where: { id: receiverWalletId },
    data: { balance: { increment: amount } }
  }),
  prisma.transaction.update({
    where: { id: transactionId },
    data: { status: 'COMPLETED', processedAt: new Date() }
  })
])

// Equivalent SQL (runs as a single transaction):
BEGIN;
  UPDATE wallets SET balance = balance - 100 WHERE id = 'sender-id';
  UPDATE wallets SET balance = balance + 100 WHERE id = 'receiver-id';
  UPDATE transactions SET status = 'COMPLETED' WHERE id = 'tx-id';
COMMIT;  -- or ROLLBACK if any step fails
```

If the server crashes after debiting the sender but before crediting the receiver, PostgreSQL automatically rolls back the entire transaction. The sender's balance is restored. No money is lost. This is ACID compliance.

7.2 Paginated Transaction History

```
// Prisma code:
const [transactions, total] = await Promise.all([
  prisma.transaction.findMany({
    where: {
      OR: [
        { senderWallet: { userId } },      // transactions I sent
        { receiverWallet: { userId } }     // transactions I received
      ]
    },
    skip: (page - 1) * limit,             // e.g. page 3, limit 20 → skip 40
    take: limit,                           // return 20 rows
    orderBy: { createdAt: 'desc' }         // newest first
  }),
  prisma.transaction.count({ where })      // total count for pagination metadata
])

// Both queries run in parallel (Promise.all), not sequentially
```

7.3 Refresh Token Rotation

```
// Atomic: revoke old token AND create new one simultaneously
await prisma.$transaction([
  prisma.refreshToken.update({
    where: { token: oldToken },
    data: { revoked: true }
  }),
  prisma.refreshToken.create({
    data: { userId, token: newToken, expiresAt }
  })
])

// If this fails halfway, neither change persists.
// The old token remains valid, the new token is not created.
// User can retry without being locked out.
```

8. Security Layers

PayFlow implements defence-in-depth — multiple independent layers of security so that breaking one doesn't compromise the system.

Layer	Tool	What It Protects Against
Password hashing	bcrypt (12 rounds)	Database theft — attackers can't recover passwords
JWT expiry	15 min access tokens	Token theft — stolen tokens expire quickly
Refresh token DB storage	PostgreSQL	Can't revoke pure JWT — DB storage enables revocation
Token rotation	Custom auth service	Token replay attacks after logout
Input validation	Zod	Injection attacks, malformed data, type confusion
Rate limiting (auth)	express-rate-limit (10/15min)	Brute-force password guessing
Rate limiting (global)	express-rate-limit (100/15min)	DDoS, API scraping
HTTP security headers	Helmet	XSS, clickjacking, MIME sniffing
CORS	cors	Unauthorised cross-origin requests
Ownership check	Wallet userId verification	User A cannot access User B's wallets
Request size limit	express.json limit 10kb	Memory exhaustion attacks
Idempotency keys	UUID + DB unique constraint	Double-charge from retries

9. What to Add Next (To Impress Even More)

The current project is already impressive for a portfolio. Here are additions that show progressively deeper engineering maturity.

Intermediate Additions

- Currency conversion — integrate an exchange rate API (fixer.io is free) to allow cross-currency transfers
- Email notifications — send an email on transfer completion using Nodemailer + free Gmail SMTP
- Webhook system — let external apps subscribe to transaction events (like Stripe webhooks)
- Soft deletes — add deleted_at column and never hard-delete data (financial audit requirement)
- Request ID middleware — add a unique X-Request-ID header to every response for tracing

Advanced Additions

- Spend limits — allow users to set daily/monthly spend limits per wallet
- Transaction reversal — implement a reversal flow with its own audit trail
- KYC flow — add document upload and verification status progression
- Admin dashboard API — separate admin routes for viewing all users and transactions
- Database read replicas — route read queries to a replica, writes to primary

DevOps Additions

- GitHub Actions CI/CD — run tests automatically on every push
- Deploy to Railway.app or Render.com — both free tiers, both support PostgreSQL and Redis
- Sentry integration — real-time error monitoring with stack traces
- API versioning strategy — document how v2 would be introduced

10. How to Talk About This in an Interview

Knowing the code is one thing. Being able to explain your decisions confidently is what gets you hired. Here are the key talking points.

When asked: 'Tell me about this project'
I built a production-grade FinTech REST API that handles multi-currency wallets
and payment transfers. I focused on the architectural decisions that real payment

systems need — idempotent transactions to prevent double-charges, async queue processing with automatic retries for reliability, atomic database transactions to prevent race conditions, and refresh token rotation for security.

When asked: 'Why did you use a queue for transfers?'

In a real payment system, you never process transfers synchronously in the HTTP request because if the server crashes mid-transfer, you lose state. With BullMQ, the job persists in Redis even through server restarts. The worker retries failed jobs automatically with exponential backoff — the same pattern Stripe uses.

When asked: 'What is idempotency and why does it matter?'

Idempotency means calling an operation multiple times has the same effect as calling it once. In payments, if a user's network drops after sending a transfer request, their app might retry. Without idempotency, Bob gets paid twice. We solve it with a unique Idempotency-Key header — if we've seen that key before, we return the original result. This is exactly how Stripe's API works.

When asked: 'How do you handle authentication?'

We use JWT with two tokens — a short-lived access token (15 minutes) and a longer-lived refresh token (7 days). The refresh token is stored in the database so it can be revoked on logout — solving the classic JWT revocation problem. We also implement rotation: every refresh issues a new token pair and revokes the old one, limiting the blast radius if a refresh token is ever stolen.

PayFlow API — Built for Learning, Designed for Production

34 Tests Passing • 4 Modules • 12 Endpoints • 4 DB Tables